



# CSS Flexbox: Diseño Flexible para el Desarrollo Web Moderno

Una guía completa para dominar el sistema de diseño más versátil de CSS. Aprenderás desde los conceptos básicos hasta técnicas avanzadas con ejemplos prácticos.

# ¿Qué es Flexbox y Por Qué Usarlo?

Flexbox (Flexible Box Layout) es un modelo de diseño en CSS que permite organizar elementos HTML de forma eficiente y predecible, incluso cuando su tamaño es desconocido o dinámico.

A diferencia de los modelos tradicionales, Flexbox distribuye el espacio de manera inteligente y alinea los elementos fácilmente en cualquier dirección.

## Ventajas Principales

- Alineación simplificada vertical y horizontal
- Distribución automática del espacio disponible
- Adaptabilidad a diferentes tamaños de pantalla
- Orden de elementos independiente del HTML

# Creando un Contenedor Flex

Para empezar a utilizar Flexbox, primero debemos crear un contenedor flex aplicando la propiedad `display: flex` a un elemento padre.

```
/* HTML */
<div class="parent">

  <div>Div 1</div>

  <div>Div 2</div>

  <ul>
    <li>List Item 1</li>
    <li>List Item 2</li>
    <li>List Item 3</li>
  </ul>

</div>
```

```
.parent {
  background-color: ghostwhite;
  display: flex;
}

div, ul {
  margin: .5rem;
  padding: .5rem;
}

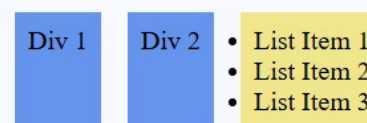
div div {
  background-color: cornflowerblue;
}

ul {
  background-color: khaki;
}
```

Desactivado



Activado: automáticamente se desplaza de manera horizontal nuestro contenido gracias a flex.



# Propiedades Principales del Contenedor Flex

## flex-direction

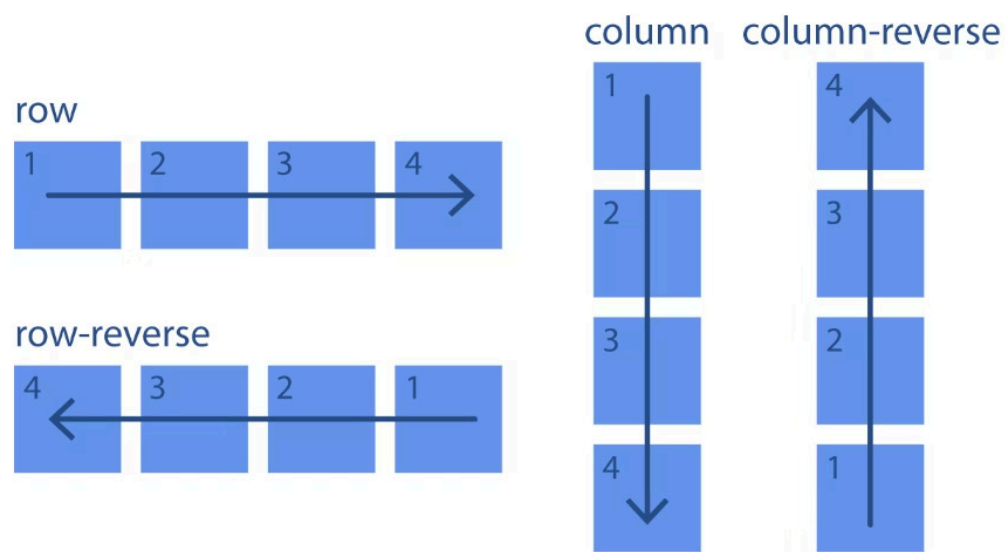
Define la dirección principal de los ítems dentro del contenedor:

- row (predeterminado): de izquierda a derecha
- row-reverse: de derecha a izquierda
- column: de arriba hacia abajo
- column-reverse: de abajo hacia arriba

## justify-content

Alinea los ítems a lo largo del eje principal:

- flex-start: al inicio
- flex-end: al final
- center: en el centro
- space-between: distribuidos con espacio entre ellos
- space-around: distribuidos con espacio alrededor
- space-evenly: espacio uniforme entre todos



### flex-start



### flex-end



### center



### space-between



### space-around



### space-evenly



# Propiedades de Alineación del Contenedor Flex

## align-items

Alinea los ítems en el eje secundario (perpendicular al eje principal):

- `stretch` (predeterminado): estira los ítems
- `flex-start`: al inicio del eje
- `flex-end`: al final del eje
- `center`: centrados en el eje
- `baseline`: alineados por la línea base del texto

## gap

Define el espacio entre los ítems:

```
.contenedor {  
  display: flex;  
  gap: 20px; /* Espaciado uniforme */  
  /* También se puede usar: */  
  /* row-gap: 10px; */ /* Espaciado entre filas */  
  /* column-gap: 15px; */ /* Espaciado entre columnas */  
}
```

### stretch



### flex-start



### flex-end



### center



### baseline



gap: 10px



# Propiedades de los Ítems Flex

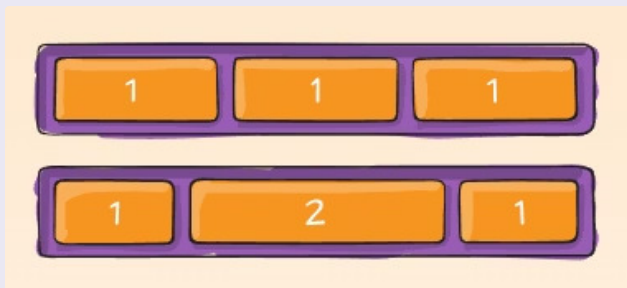
Mientras que las propiedades del contenedor definen el comportamiento general, las propiedades de los ítems permiten personalizar cada elemento individualmente.



## flex-grow

Define la capacidad de un ítem para crecer si hay espacio disponible. Un valor más alto significa mayor capacidad de crecimiento en proporción a otros ítems.

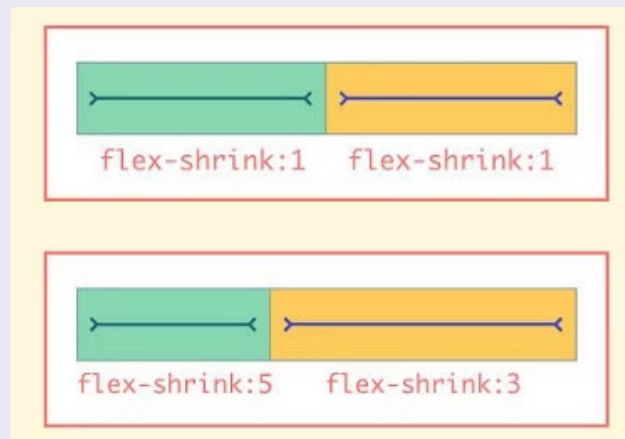
```
.item-2 { flex-grow: 2; }
```



## flex-shrink

Determina cuánto puede encogerse un ítem si el espacio es insuficiente. Un valor más alto significa mayor capacidad de encogimiento.

```
.item { flex-shrink: 2; }
```



## flex-basis

Establece el tamaño inicial del ítem antes de que se apliquen flex-grow o flex-shrink. Puede ser una longitud o auto en este caso el basis es de 200px en el 4 ítem.

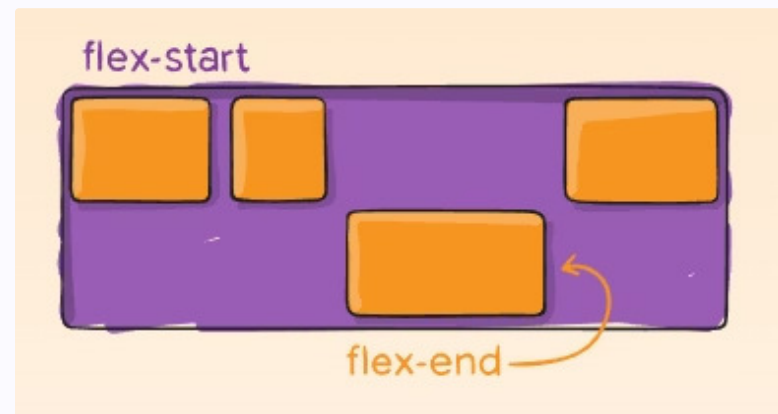
```
.item { flex-basis: 200px; }
```



# Alineación Individual con align-self

La propiedad `align-self` permite sobrescribir la alineación definida por `align-items` para ítems específicos.

```
.contenedor {  
  display: flex;  
  align-items: center; /* Alineación por defecto */  
  height: 200px;  
  border: 2px solid #403ccf;  
}  
  
.item:nth-child(2) {  
  align-self: flex-start; /* Sobrescribe para el segundo ítem */  
}  
  
.item:nth-child(4) {  
  align-self: flex-end; /* Sobrescribe para el cuarto ítem */  
}
```



Esta propiedad acepta los mismos valores que `align-items`: `stretch`, `flex-start`, `flex-end`, `center` y `baseline`.

# Ejercicio Práctico: Aplicando Flexbox

Es hora de poner en práctica lo aprendido. Construye una página web básica aplicando las propiedades de Flexbox para lograr los siguientes diseños:

- 

### Estructura Semántica

Crea una página con las etiquetas semánticas `<header>`, `<nav>`, `<main>` y `<footer>`.
- 

### Header Flexible

Usa Flexbox en el `<header>` para alinear el logo y el menú de navegación en la misma línea, uno a cada extremo.
- 

### Artículos Adaptativos

Dentro del `<main>`, crea una sección con tres artículos. Estos deben distribuirse usando `justify-content` y deben ajustarse a la línea siguiente con `flex-wrap` si no hay espacio suficiente.
- 

### Cajas Centradas

En otra sección dentro del `<main>`, muestra un conjunto de cajas. Estas deben estar centradas tanto horizontal como verticalmente usando `align-items` y `justify-content`.
- 

### Footer Espaciado

El `<footer>` debe contener varios elementos (ej. enlaces, derechos de autor) alineados con `space-between` para distribuir el espacio uniformemente.

Como debería verse!

Mi Sitio

Inicio Servicios Contacto

Artículo 1

Artículo 2

Artículo 3

Artículo 4

A

B

C

© 2025 Mi Sitio

Facebook Twitter

Este ejercicio te ayudará a consolidar los conceptos de contenedor y ítems flex, así como las propiedades de alineación y distribución de espacio.



# Codigo terminado

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Ejemplo Flexbox</title>
  <link rel="stylesheet" href="style.css">
</head>

<body>
  <!-- Encabezado con logo y navegación -->
  <header class="header">
    <h1 class="logo">Mi Sitio</h1>
    <nav class="nav">
      <ul>
        <li><a href="#">Inicio</a></li>
        <li><a href="#">Servicios</a></li>
        <li><a href="#">Contacto</a></li>
      </ul>
    </nav>
  </header>

  <!-- Contenido principal -->
  <main>
    <!-- Sección de artículos -->
    <section class="articulos">
      <article>Artículo 1</article>
      <article>Artículo 2</article>
      <article>Artículo 3</article>
      <article>Artículo 4</article>
    </section>

    <!-- Sección de cajas centradas -->
    <section class="cajas">
      <div class="box">A</div>
      <div class="box">B</div>
      <div class="box">C</div>
    </section>
  </main>

  <!-- Pie de página -->
  <footer class="footer">
    <p>&copy; 2025 Mi Sitio</p>
    <div class="social">
      <a href="#">Facebook</a>
      <a href="#">Twitter</a>
    </div>
  </footer>
</body>

</html>
```

```
body {
  font-family: Arial, sans-serif;
  background: #f5f5f5;
  line-height: 1.6;
}
/* ----- HEADER ----- */
.header {
  display: flex; /* Habilitamos flexbox */
  justify-content: space-between; /* Logo a la izquierda, menú a la
derecha */
  align-items: center; /* Centrados verticalmente */
  padding: 10px 20px;
  background: #333;
  color: white;
}
.nav ul {
  display: flex;
  list-style: none;
  gap: 15px; /* espacio entre links */
}
.nav a {
  color: white;
  text-decoration: none;
  font-weight: bold;
}

/* ----- SECCIÓN ARTÍCULOS ----- */
.articulos {
  display: flex;
  flex-wrap: wrap; /* Los artículos pueden saltar de línea */
  justify-content: space-around; /* Espacio uniforme entre ellos */
  gap: 15px;
  padding: 20px;
}
.articulos article {
  background: #4a90e2;
  color: white;
  flex: 1 1 200px; /* Crecen, encogen, mínimo 200px */
  padding: 20px;
  text-align: center;
  border-radius: 8px;
}

/* ----- SECCIÓN CAJAS ----- */
.cajas {
  display: flex;
  justify-content: center; /* Centrado horizontal */
  align-items: center; /* Centrado vertical */
  gap: 20px;
  height: 200px;
  background: #ddd;
  margin: 20px;
}
.box {
  background: #50e3c2;
  color: #333;
  font-size: 24px;
  font-weight: bold;
  padding: 30px;
  border-radius: 10px;
}
/* ----- FOOTER ----- */
.footer {
  display: flex;
  justify-content: space-between; /* Elementos a los extremos */
  align-items: center;
  padding: 15px 20px;
  background: #222;
  color: white;
}
.footer .social {
  display: flex;
  gap: 10px;
}
.footer a {
  color: #50e3c2;
  text-decoration: none;
}
```

# Casos de Uso Prácticos

1

## Barra de navegación

Ideal para crear menús horizontales o verticales con elementos distribuidos uniformemente o agrupados según se necesite.

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
}
```

2

## Diseño de tarjetas

Perfecto para distribuir tarjetas de contenido en una cuadrícula adaptable, donde las tarjetas pueden tener diferentes alturas.

```
.cards-container {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 20px;  
}
```

3

## Formularios responsivos

Permite crear formularios que se adaptan a diferentes tamaños de pantalla, cambiando de horizontal a vertical según sea necesario.

```
.form {  
  display: flex;  
  flex-direction: column;  
  gap: 15px;  
}  
  
@media (min-width: 768px) {  
  .form {  
    flex-direction: row;  
  }  
}
```

Flexbox ha revolucionado la manera en que diseñamos interfaces web, permitiendo implementar diseños complejos de forma más sencilla e intuitiva.

# Consejos y Mejores Prácticas

## Combina Flexbox con Grid

Usa CSS Grid para el layout general de la página y Flexbox para componentes más pequeños. Ambos son complementarios, no competidores.

## Cuidado con el anidamiento

Evita anidar demasiados contenedores flex, ya que puede complicar el mantenimiento y afectar el rendimiento.

## Usa la propiedad flex abreviada

En lugar de definir flex-grow, flex-shrink y flex-basis por separado, usa la propiedad abreviada flex para un código más limpio.

## Recursos Adicionales

- [CSS-Tricks: Guía completa de Flexbox](#)
- [Flexbox Froggy: Juego para aprender Flexbox](#)
- [MDN Web Docs: Diseño de Caja Flexible CSS](#)

Dominar Flexbox te dará una gran ventaja en el desarrollo front-end moderno, permitiéndote crear interfaces más robustas y adaptables a cualquier dispositivo.